



Universidad Nacional de Colombia

Sede Medellín

Introducción a la Inteligencia Artificial – 3010476

Trabajo Práctico 1 - Valor: 10%

Sistemas Expertos, Lógica Difusa y Ontologías

Profesor Jaime Alberto Guzmán Luna

Fecha de Entrega: 29 de Septiembre, **Hora de cierre:** 9:50 am.

Fecha de Sustentación: 29 - 1 de Septiembre en el horario de clase. Componentes: Sistemas Expertos + Lógica Difusa + Ontologías

OBJETIVO

Diseñar e implementar un sistema híbrido de inteligencia artificial que integre tres paradigmas clásicos: sistemas expertos basados en reglas, lógica difusa y representación del conocimiento mediante ontologías, para resolver un problema complejo del mundo real, permitiendo la toma de decisiones contextualizadas y enriquecidas semánticamente.

ENUNCIADO DEL PROBLEMA

1. DESCRIPCIÓN GENERAL

El grupo de trabajo deberá seleccionar un dominio de aplicación realista y proponer una solución inteligente basada en la combinación de:

- Sistema experto para tomar decisiones lógicas basadas en reglas precisas (utilizando Expertá).
- Sistema de lógica difusa para modelar incertidumbre y valores intermedios del entorno (usando Scikit-Fuzzy).
- Ontología y su razonamiento semántico para representar e inferir entidades y relaciones del dominio (en RDF/RDFS, usando RDFLib y OWL-RL).

2. COMPONENTES DEL SISTEMA HÍBRIDO

2.1. Sistema Experto (Expertá)

- Definir clases de Hechos: Se deben definir al menos unas 5 clases de hechos
- Definir reglas basadas en el conocimiento del dominio:
 - Se deben definir al menos 15 reglas que hagan uso de las clases de hechos anteriormente definidas y sus instancias.
 - Utilizar prioridades y las técnicas de control de ejecución así:
 - Utilizar al menos 3 niveles de prioridad.
 - Implementar los tres mecanismos diferentes para evitar conflictos entre reglas (Recencia, Specificity, Saliencia), y explicarlos detalladamente en el documento.
 - Implementar al menos un mecanismo para evitar conflictos bucles infinitos (No-Loop ó Lock-Fact).
- Definir la base de hechos:
 - Definir mínimo 40 hechos.

2.2. • Sistema de lógica difusa (Scikit-Fuzzy)

- Definir al menos 3 variables difusas del dominio
 - Para cada variable debes tener mínimo 3 valores difusos y asociarles sus funciones de pertenencia.
 - Para cada variable definir su universo del discurso.
 - Las funciones de pertenencias deben incluir obligatoriamente en el total de todos sus casos las siguientes funciones: triangulares, trapezoidales y Gaussianas.
 - Hacer uso de dos casos de modificadores (ejemplo “ligeramente”, “muy”, etc.) para lo cual aplicarán su respectiva fórmula detallada en clase.
- Definir al menos 9 reglas difusas que incluyan casos con los operadores: Complemento (negación), Intersección (AND) y Unión (OR).
- Implementar un proceso de defuzzificación justificado (centroide, bisector, promedio, etc).

2.3. Ontología y Razonamiento Semántico (RDFLib y OWL-RL)

- Implementación de la Ontología del dominio utilizando RDF y RDFS
 - El lenguaje de serialización de la ontología final desarrollada será en Turtle.
 - Implementar al menos 10 clases (rdfs:Class) y 5 relaciones jerárquicas (rdfs:subClassOf).
 - Crear al menos 10 propiedades RDF originales con restricciones: dominios (rdfs:domain) y rangos (rdfs:range) correctamente definidos.
 - Incluir al menos un caso de jerarquía de propiedad (rdfs:subPropertyOf)

- Incluir como rangos valores tanto de tipo clase (URIs) y otros tipo Literales con sus respectivos tipados usando el XML-Schema. Incluir el uso de clases o propiedades provenientes de vocabularios definidos y aceptados en la literatura (FOAF, Dublin Core, etc.).
 - Instanciar al menos 4 individuos por clase
 - Cada una de las clases y propiedades deberán escribir su significado haciendo uso de la propiedad `rdfs:label`
- Aplicar razonamiento sobre la ontología RDFS anteriormente desarrollada usando `DeductiveClosure(RDFS Semantics)` de `owlrl`.
 - Completar de ser necesario la ontología desarrollada para realizar lo siguiente:
 - Documentar al menos dos casos de generación de nuevos hechos asociados a una jerarquía de clases (ver caso 1 de las diapositivas)
 - Documentar la asociación de un caso de miembro de una clase desde el dominio o rango de sus propiedades (ver caso 2 de las diapositivas)
 - Documentar un caso de nuevos hechos desde la relación `subproperty` (ver caso 3 de las diapositivas)
 - Todos los tres requerimientos anteriores, deben ser generados al usar el razonador `OWL-RL` usando `DeductiveClosure(RDFS Semantics)` sobre la ontología final de su dominio.
 - Comparar el grafo antes y después del razonamiento para evidenciar nuevas afirmaciones inferidas automáticamente.
 - Adicionalmente a lo anteriormente requerido se debe evidenciar al menos 4 nuevos hechos inferidos generados como resultado del razonamiento.

2.4. Enfoque propuesto para la Integración

Se va a Implementar un enfoque donde:

- El componente de la ontología alimentará al Sistema Experto.
 - Para tal fin se va a consultar la ontología RDF y se van a traducir las inferencias semánticas obtenidas con el razonador `OWL-RL` (Grafo resultante de razonamiento semántico) como hechos iniciales del sistema experto. Para ello se deberá implementar el traductor respectivo para alimentar el sistema experto.

Traductor Ontología → Sistema Experto

El traductor **debe consultar dinámicamente el grafo RDF resultante** (ontología + inferencias del razonador `OWL-RL`) y generar a partir de él los hechos iniciales del sistema experto. No es válido declarar manualmente en el código del sistema experto los hechos correspondientes a los datos observados en la ontología, utilizando valores fijos (por ejemplo, `engine.declare(Padre(padre='Juan', hijo='Pedro'))`); el código del traductor no debe contener nombres ni valores propios de un caso específico. Cada grupo es libre de implementar la lectura del grafo con el método que prefiera, siempre que el traductor realmente lea y traduzca la información del grafo, y no se limite a transcribir a mano lo que se observó en la ontología en un momento dado.

Caso de prueba obligatorio: se debe agregar un individuo o relación nueva directamente en el archivo de la ontología, sin modificar el código del traductor ni del sistema experto, volver a ejecutar el traductor, y evidenciar que el hecho nuevo se refleja automáticamente como **Fact** en el sistema experto y es procesado correctamente por las reglas correspondientes. Si para reflejar el cambio es necesario editar el código Python, el traductor no cumple con lo solicitado.

- El componente de la Lógica Difusa igualmente alimentará al Sistema Experto.
 - Para tal fin se usarán los resultados de la lógica difusa como condiciones adicionales para activar reglas en el sistema experto. Es decir, se deben tener en cuenta dichos resultados en algunas de las precondiciones de las reglas del Sistema Experto.
- El sistema Experto luego de recibir la información de los anteriores componentes (Ontología y Lógica difusa) procederá a realizar las inferencias respectivas.

Nota Importante sobre la Ontología y el Sistema Experto:

Todos los hechos generados en la ontología deben ser traducidos sin excepción al Sistema Experto como hechos iniciales. No se aceptará que se construya una ontología extensa para luego utilizar únicamente una parte reducida en el Sistema Experto. Esto implica que deberán existir reglas adicionales que incluyan en sus precondiciones o efectos estos hechos.

La evaluación de esta actividad verificará que:

- El **grafo resultante** después del razonamiento semántico sea la **fuentes completa de hechos** que alimentan el Sistema Experto.
- **Cada triple relevante (clases, propiedades, individuos e inferencias)** se traduzca a hechos del motor de reglas, manteniendo la coherencia semántica.
- El Sistema Experto haga uso explícito de dichos hechos en la activación de reglas e inferencias posteriores.

Es decir, la ontología no debe quedar como una base de datos aislada; **todo el conocimiento representado y deducido en ella debe reflejarse en el Sistema Experto.**

3. ENTREGABLES

3.1. Documento de diseño (PDF):

- Descripción detallada de cada componente: sistema experto, lógica difusa y ontología
- Particularmente:
 - Gráficas de funciones de pertenencia utilizadas en el componente

de lógica difusa.

- Diagrama de clases y documentación de los tres casos de generación de nuevo conocimiento solicitados en el módulo de Ontologías.
- Enlace del vídeo tipo pitch de sustentación que se detalla en el numeral 4 (sustentación).

3.2. Código fuente documentado en Python (Google Colab):

- Código fuente separado por módulos: experto, difuso y ontología.
- Comentarios por función y explicación del flujo general

4. SUSTENTACIÓN

- Se realizará de manera virtual.
- Se deberá implementar un vídeo tipo pitch (duración máx. 5 minutos) con la demostración del sistema funcional, la explicación clara del problema y la solución híbrida implementada.
 - Todos los integrantes deben participar en el video.
 - El video se presentará durante la Sustentación, para lo cual deberán tener un buen equipo con internet para su presentación virtual.
 - El video deberá ser subido a YouTube y en el documento de diseño (PDF) a entregar incluir el enlace de este (ver numeral 3.1).
 - Este video deberá estar disponible desde la fecha de entrega del trabajo indicada al inicio de este documento.
- Se realizará una sesión de preguntas (5 min el pitch y 5 min de preguntas).
 - Todos los integrantes deben estar presentes en la sustentación y se podrá seleccionar uno cualquiera para responder preguntas.

5. METODOLOGÍA DE EVALUACIÓN

●Código y documentación: 65%

- Módulo de Sistemas Expertos: 25%
- Módulo de Lógica Difusa: 15%
- Módulo de Ontología: 25%

● Sustentación: 35%

- Video tipo pitch: 20%
- Atención a preguntas: 15 %

6. MATERIAL A ENTREGAR

Se deberá entregar en Google Classroom un archivo ZIP (con el siguiente nombre del archivo: practica1-grupo-**XX**-equipo-**YY**.zip, donde **XX** representa el número del grupo (1 o 2) y **YY** el número del equipo) que contenga lo siguiente:

- **Documento de diseño:** Descripción detallada de cada módulo (experto, difuso, ontología), junto con la justificación de las decisiones tomadas en cada parte del programa y **el enlace al video de sustentación tipo pitch.**
- **Código Fuente:** Todo el programa en Python, debidamente organizado y documentado.

ANEXO 1

INSTRUCCIONES GENERALES PARA LA PRESENTACIÓN DE LOS TRABAJOS

- Los trabajos serán presentados en grupos de 4 personas definidos al inicio del semestre (ver grupos en el material de la clase 2).
- No se admiten trabajos similares **(incluyendo el planteamiento del dominio)**, en caso de presentarse programas con códigos similares serán anulados. La totalidad del trabajo solicitado al alumno, en esta evaluación práctica, deberá ser original y propio del autor que lo presenta. El estudiante es responsable de evitar que su material evaluable (código, solución al problema, memorias, etc.) sea accesible a estudiantes de otros grupos. En caso de que se detecten copias por incumplimiento de estas reglas, por acción o inacción, la sanción afectará a todos los estudiantes involucrados: quienes copien y quienes hayan sido copiados.
- Los integrantes de cada grupo deberán ser capaces de explicar y responder preguntas sobre el trabajo realizado durante la sustentación del trabajo.
- La evaluación de la práctica está asociada con la sustentación respectiva programada. Para tal fin, la persona que no asista a la sustentación, se considerará que no presentó la práctica en su totalidad (**“no se tendrá en cuenta el código, ni la documentación, ni el video tipo pitch para la evaluación”**).
- El plazo de entrega del material asociado a la práctica es estricto y se indica al inicio de este documento. No se admiten trabajos después de esta fecha. No se considera como entrega aquella que solo contiene código o solo contiene el vídeo o solo contiene el documento de diseño o cualquier forma incompleta del trabajo.
- El código entregado en el archivo .zip en Google Classroom, debe ser el mismo que se ilustra en el vídeo y se explica en el documento de diseño. En caso de no cumplirse, se considera que el código no fue entregado y, por tanto, no se realizó la práctica por parte del grupo.

NOTA1: Si en la fecha de la entrega de la práctica no se entrega el video y luego en sustentación no se presenta el video, el trabajo de la práctica se considerará no entregado.

NOTA 2: La sustentación del trabajo se realizará en la fecha indicada y se realizará mediante el vídeo entregado y las preguntas orales que se realizarán a los integrantes de cada equipo después de la presentación del pitch.

NOTA 3: La redacción de este documento de práctica puede presentar algunos errores o la ausencia de algunos requisitos específicos, los cuales deberán ser informados por parte de los estudiantes al profesor, para ser aclarados de manera colectiva durante la clase. Estas aclaraciones harán parte de los requisitos de la práctica.